

Desarrollo de una celda reconfigurable de CGRA

Duarte Sanchez David, Jimenez Ulate Eddy, Ruiz Rivera Jeffrey, Esquivel Arias Fabian Zúñiga Ramírez Jose Eduardo
Grupo de 5

{davidduarte28, e.jimenez.10, yr1ruiz, fabiesqui22, je.zuniga}@estudiantec.cr

I. INTRODUCCIÓN

Las arquitecturas reconfigurables han sido muy exitosas en la industria debido a que la evolución del software ha presentado una rápida evolución, con diversas aplicaciones, distintas necesidades de usuario y progreso científico, por lo que el hardware no se puede adaptar en la misma velocidad al software [1]. Sistemas de propósito específico pueden sufrir de obsolescencia rápida y si el precio del desarrollo y producción del hardware es alto, se vuelve inviable económicamente [1].

Actualmente, arquitecturas como las FPGA (*Field Programmable Gate Arrays*) han sido ampliamente adoptadas en diversas aplicaciones debido a su flexibilidad y capacidad de personalización, tomando fuerza en Computación de alto desempeño (HPC) por sus siglas en inglés. Sin embargo, las FPGAs presentan ciertas limitaciones como tiempos de configuración lentos, largos tiempos de compilación y aunque es algo relativo, bajas frecuencias de operación [2].

Debido a lo anterior, se ha tomado un interés en HPC en utilizar arquitecturas como los CGRAs (*Coarse-Grained Reconfigurable Arrays*) que ofrecen una solución intermedia entre los FPGAs y los procesadores de propósito general (GPP). Los CGRAs son arquitecturas de hardware que conceptualmente permiten que el silicio sea maleable y su funcionalidad pueda reconfigurarse de manera dinámica [2]. Los CGRA tienen la ventaja de ser uno o dos órdenes de magnitud más eficientes que las FPGAs y más de 2 a 3 órdenes de magnitud más eficientes que los GPPs [1].

El desarrollo de una celda reconfigurable de CGRA es de gran importancia por su relevancia no solo en HPC, si no también en su adopción en aplicaciones de *Machine-learning*, procesamiento de imágenes y visión por computadora, comunicaciones y otras áreas [3]. Por lo tanto el objetivo de este trabajo es el desarrollo de un CGRA heterogéneo, que posea tentativamente celdas de ruteo de datos, celdas con bancos de memoria distribuida, celdas de procesamiento escalar y celdas de procesamiento vectorial, con la capacidad de ejecutar programas utilizando el set de instrucciones RISC-V, con la capacidad de ejecutar lógica de ocho, 16 y 32 bits.

La importancia de utilizar este set de instrucciones es que este es un estándar abierto, facilitando su adopción debido a que no requiere pagos por licencias. Este es un set de instrucciones basado en la simplicidad, haciendo que el diseño de hardware sea sencillo y una implementación más directa [4].

II. ANTECEDENTES Y TRABAJO RELACIONADO

En los últimos años, las CGRA han ganado un protagonismo significativo como aceleradores de hardware debido a su capacidad para ofrecer un equilibrio excepcional entre la flexibilidad del software y el alto rendimiento del hardware de aplicación específica [1], [2]. A diferencia de las arquitecturas de grano fino, las CGRAs operan a nivel de palabra o bloque, lo que reduce sustancialmente la sobrecarga de reconfiguración y enrutamiento, haciéndolas particularmente atractivas para aplicaciones eficientes en el borde (*edge computing*). El desarrollo e investigación de estas arquitecturas ha sido fuertemente impulsado por la adopción del repertorio de instrucciones (ISA) de código abierto RISC-V [4], cuya naturaleza extensible facilita la creación de Sistemas en Chip (SoC) heterogéneos altamente personalizados.

Investigaciones recientes han demostrado las ventajas de integrar estrechamente núcleos RISC-V con arreglos CGRA para superar el cuello de botella tradicional de Von Neumann. Por ejemplo, [5] exploran la integración de un acelerador CGRA acoplado a un núcleo de alto rendimiento RISC-V CVA6 enfocado en ecosistemas cloud-edge, mientras que [3] proponen DVHetero, un entorno de trabajo integral para el diseño y la validación de SoCs heterogéneos basados en esta combinación.

A nivel de implementación práctica, estas arquitecturas se materializan en esfuerzos de código abierto de vanguardia; repositorios como *risc-v-cgra* (desarrollado por ECASLab) investigan estas integraciones a nivel de sistema, mientras que proyectos como OpenEdgeCGRA (ESL-EPFL) proporcionan plataformas open-hardware completas. En este último, la arquitectura se compone de una malla bidimensional configurable de Elementos de Procesamiento (PEs) o celdas reconfigurables que interactúan en forma de toroide y comparten registros para maximizar la paralelización de instrucciones con una sobrecarga de memoria mínima. Otro proyecto destacable como OpenCGRA desarrollado por el PNNL (*Pacific Northwest National Laboratory*) el cual genera procesadores reconfigurables de forma parametrizable, de esta manera, permite diseñar el hardware por código como el tamaño de la malla, el tipo de celdas, cómo se comunican entre ellas, luego se prueba el funcionamiento usando Python y Verilator y finalmente se exporta el diseño a un código de diseño de hardware estándar (HDL) como Verilog.

Para que este tipo de sistemas alcance su máximo potencial, el diseño interno de la celda reconfigurable individual (PE)

y las herramientas de compilación asociadas son críticos [6]. Destacan la necesidad de un mapeo consciente del contexto de memoria (context-memory aware mapping) para asegurar una aceleración eficiente desde el punto de vista energético, subrayando que la latencia en el acceso a datos dicta el rendimiento global. En este marco, el diseño y desarrollo de una nueva celda reconfigurable de CGRA se fundamenta en la necesidad de optimizar las interconexiones locales, el acceso determinista a operandos intermedios y la versatilidad de sus operaciones a nivel de ciclo de reloj, retos que continúan siendo áreas de activa exploración en la frontera del diseño de aceleradores heterogéneos.

III. ANÁLISIS DE SOLUCIONES EXISTENTES (SOTA)

Existen distintos tipos de CGRAs, tipos como el ADRES y el HyCUBE son de tipo homogéneo, mientras que el Plasticine posee dos tipos de PEs [7].

El diseño de soluciones óptimas requiere un escrutinio riguroso de las alternativas vigentes en la literatura científica. En esta sección se evalúan las fortalezas y debilidades de los enfoques predominantes en el estado del arte (SOTA), delimitando sus escenarios de viabilidad técnica.

III-A. Estudio Comparativo de Propuestas Vigentes

Las soluciones actuales presentan compromisos fundamentales (*trade-offs*) entre flexibilidad, eficiencia y costo de implementación:

- **Enfoque Basado en Procesadores de Propósito General (GPP):** Destaca por su alta flexibilidad de programación y madurez en cadenas de desarrollo. No obstante, presenta debilidades críticas en entornos de tiempo real debido al determinismo limitado y un consumo energético ineficiente bajo cargas masivas de datos. Es viable únicamente en escenarios donde las restricciones de potencia y latencia son secundarias.
- **Enfoque Basado en Aceleradores Dedicados (ASIC):** Ofrece el máximo rendimiento por unidad de área y un consumo energético mínimo. Sin embargo, su rigidez absoluta impide la reconfiguración ante actualizaciones algorítmicas, además de acarrear costos de desarrollo inviables para producciones de bajo volumen.
- **Enfoque Basado en Lógica Programable (FPGAs / Arquitecturas Reconfigurables):** Representa un punto medio balanceado. Permite la optimización a nivel de bits y paralelismo masivo. Su principal desventaja radica en la alta complejidad de diseño, tiempos de compilación elevados y la gestión compleja de la memoria compartida.

Para ilustrar de forma concisa estos compromisos, la Tabla I resume los criterios de viabilidad tecnológica encontrados en la literatura.

III-B. Lista de Desafíos Observados

A partir del análisis sistemático del SOTA, se derivan los siguientes desafíos técnicos abiertos que justifican el desarrollo de una nueva alternativa:

Cuadro I
MATRIZ COMPARATIVA DE SOLUCIONES EN EL ESTADO DEL ARTE

Solución	Fortalezas	Debilidades	Viabilidad
Propuesta A [8]	Baja latencia, optimización espacial.	Rigidez de hardware, alto tiempo de diseño.	Dominios fijos estables.
Propuesta B [9]	Alta flexibilidad, reconfiguración ágil.	Alto consumo de recursos, <i>overhead</i> .	Entornos dinámicos no críticos.
Propuesta C [10]	Procesamiento masivo por ciclo (OP/s).	Consumo energético restrictivo.	Infraestructura fija/servidores.

- D1. Rigidez Estructural:** Incapacidad de los aceleradores actuales para adaptarse a variaciones dinámicas en el dominio de los datos sin comprometer severamente el *timing*.
- D2. Eficiencia de Recursos Embebidos:** Degradación del rendimiento de procesamiento por unidad de área o celda cuando se escalan los tamaños de datos.
- D3. Determinismo Temporal:** Cuellos de botella en la latencia de interconexión que impiden garantizar restricciones de tiempo real estricto.

IV. PROPUESTA DE LA SOLUCIÓN Y ARQUITECTURA DE SEGUNDO NIVEL

IV-A. Descripción General de la Solución y Alcance

El presente proyecto propone el diseño de una celda reconfigurable heterogénea para un arreglo de compuertas reconfigurables de grano grueso (CGRA) que supera las limitaciones fundamentales de las arquitecturas homogéneas dominantes en el estado del arte. A diferencia de los enfoques tradicionales donde todos los elementos de procesamiento (PEs) comparten la misma estructura funcional —generando sub-utilización ante cargas de trabajo heterogéneas [8], [9]—, la solución introduce **cuatro tipos de celdas especializadas** que coexisten dentro del mismo *fabric* reconfigurable:

1. **Celdas de enrutamiento** con FIFOs para manejo dinámico del flujo de datos sin bloqueos.
2. **Celdas de memoria distribuida** con bancos SRAM locales para eliminar cuellos de botella de acceso centralizado.
3. **Celdas de procesamiento escalar** con ALU multi-precisión (enteros de 8, 16 y 32 bits) y soporte de bucles hardware.
4. **Celdas de procesamiento vectorial** con unidad aritmética SIMD en punto flotante de 32 bits para 4 u 8 elementos simultáneos.

La Tabla II resume cómo la arquitectura propuesta aborda cada uno de los desafíos identificados en el análisis del estado del arte.

IV-A1. Novedad Arquitectónica: La contribución principal reside en la combinación inédita de: (a) enrutamiento como celda de primera clase con FIFOs dedicados, (b) memoria

Cuadro II
MAPEO DE DESAFÍOS DEL SOTA A SOLUCIONES PROPUESTAS.

ID	Desafío	Solución propuesta
C1	PEs homogéneos generan sub-utilización	4 tipos de celda especializados (enrutamiento, memoria, escalar, vectorial)
C2	Overhead de enrutamiento consume > 30 % del área	Celdas de enrutamiento dedicadas con FIFOs eliminan recursos de routing de las celdas de cómputo
C3	Memoria centralizada crea cuellos de botella	Bancos SRAM distribuidos en celdas de memoria accesibles localmente
C4	Latencia de configuración RISC-V	Interfaz CSR RISC-V con DMA para carga de bitstream; configuración parcial por contexto
C5	Multi-precisión simultánea	ALU escalar con modos int8/16/32; VALU vectorial float32 en celdas separadas
C6	Overhead de bucles software	Soporte de bucles hardware (contador + predicción) en celdas escalares
C7	Verificación costosa	Metodología de programas de prueba aleatorios [11]
C8	Tiempo de reconfiguración	Multi-contexto: hasta 4 configuraciones almacenadas, seleccionable en 1 ciclo
C9	NoC como cuello de botella	Malla 2D con enlaces de 32 bits, control de flujo basado en créditos por FIFO
C10	Compilación compleja	Mapeado asistido por tipo de celda: el scheduler RISC-V clasifica operaciones por tipo antes de asignar

distribuida por celda, (c) multi-precisión entero-flotante en celdas diferenciadas, y (d) compatibilidad nativa con RISC-V (RV32IMAF) mediante registros de control y estado (CSR) y DMA en un único *fabric* heterogéneo. Ninguna de las arquitecturas revisadas —ADRES, HyCUBE, Plasticine, CGR-NPU, CGRA-RISC, DVHetero, MC-DeF— combina simultáneamente estas cuatro características.

IV-A2. Alcance del Proyecto: El alcance del presente trabajo se delimita a los siguientes entregables:

- **Diseño RTL (VHDL)** de las 4 celdas individuales más el router NoC básico.
- **Integración** de un arreglo mínimo de 2×2 celdas conectadas por NoC en malla.
- **Verificación funcional** mediante testbench exhaustivo y método de programas de prueba aleatorios.
- **Síntesis en FPGA** (target: Xilinx Artix-7 o Kintex-7) con reporte de recursos y frecuencia máxima.
- **Evaluación de rendimiento** mediante benchmarks representativos: multiplicación de matrices, FFT de 8 puntos y reducción de vectores.

Las métricas de evaluación se resumen en la Tabla III.

Cuadro III
MÉTRICAS DE EVALUACIÓN DEL PROYECTO.

Métrica	Unidad	Herramienta
Recursos FPGA	LUT, FF, DSP, BRAM	Vivado 2023.1 Implementation Report
Frecuencia máxima ($F_{\max}$)	MHz	Vivado Timing Report
Latencia por operación	ciclos	Simulación RTL (GHDL / Vivado Sim)
Throughput	OP/s, GFlop/s	Calculado a partir de $F_{\max}$ y latencia
Consumo de potencia	mW	Vivado Power Analysis (post-implementation)
Cobertura de código	%	GHDL + gcov / Vivado Simulator coverage
Speedup sobre SW	$\times$	Comparación ciclos CGRA vs. ciclos RISC-V

IV-B. Arquitectura General de Segundo Nivel — Detalle de Componentes

La arquitectura de segundo nivel del CGRA heterogéneo se organiza en una topología de malla 2×2 que integra los cuatro tipos de celdas especializadas interconectadas mediante una red en chip (NoC) con control de flujo basado en créditos. La Figura 1 presenta el diagrama de bloques de esta arquitectura.

IV-B1. Celda de Enrutamiento (Routing Cell): La celda de enrutamiento es un componente de primera clase en la arquitectura, a diferencia de los enfoques tradicionales donde el enrutamiento se realiza mediante multiplexores estáticos integrados en los PEs de cómputo. Sus características principales son:

- **FIFOs de entrada/salida:** Cada puerto cardinal (Norte, Sur, Este, Oeste) cuenta con un buffer FIFO de profundidad configurable (4–8 entradas de 32 bits) que permite absorber ráfagas de datos y desacoplar temporalmente productor y consumidor.
- **Crossbar configurable:** Una matriz de interconexión interna permite enrutar cualquier entrada a cualquier salida en un único ciclo de reloj, soportando comunicación multicast.
- **Control de flujo por créditos:** El mecanismo de créditos garantiza que no se envíen datos cuando el FIFO receptor está lleno, eliminando la pérdida de datos sin requerir señales de stall globales.
- **Configuración por contexto:** La tabla de enrutamiento se almacena en un registro de configuración seleccionable entre hasta 4 contextos, permitiendo cambio de mapeo en 1 ciclo.

Al dedicar una celda exclusivamente al enrutamiento, se libera área en las celdas de cómputo y se reduce el overhead de interconexión a menos del 15 % del área total del arreglo, comparado con el ~ 30 % reportado en arquitecturas homogéneas como HyCUBE [9].

IV-B2. Celda de Memoria Distribuida (Memory Cell): La celda de memoria distribuida resuelve el cuello de botella de acceso centralizado presente en arquitecturas como Plasticine y DVHetero. Sus componentes internos incluyen:

- **Banco SRAM local:** Un bloque de memoria de doble puerto (dual-port) implementado sobre BRAM de la FPGA, con capacidad de 1–4 KB por celda y ancho de palabra de 32 bits.
- **Controlador de acceso:** Unidad que gestiona las operaciones de lectura y escritura, soportando modos de direccionamiento directo, indirecto y con stride para patrones de acceso regulares.
- **Interfaz NoC:** Puertos de entrada/salida compatibles con el protocolo de créditos de la malla, permitiendo que celdas vecinas accedan al banco SRAM con latencia de 1–2 ciclos.
- **DMA local:** Un motor DMA simplificado permite transferencias en bloque entre la memoria principal del sistema (accedida vía la interfaz RISC-V) y el banco SRAM local sin intervención del procesador host durante la ejecución.

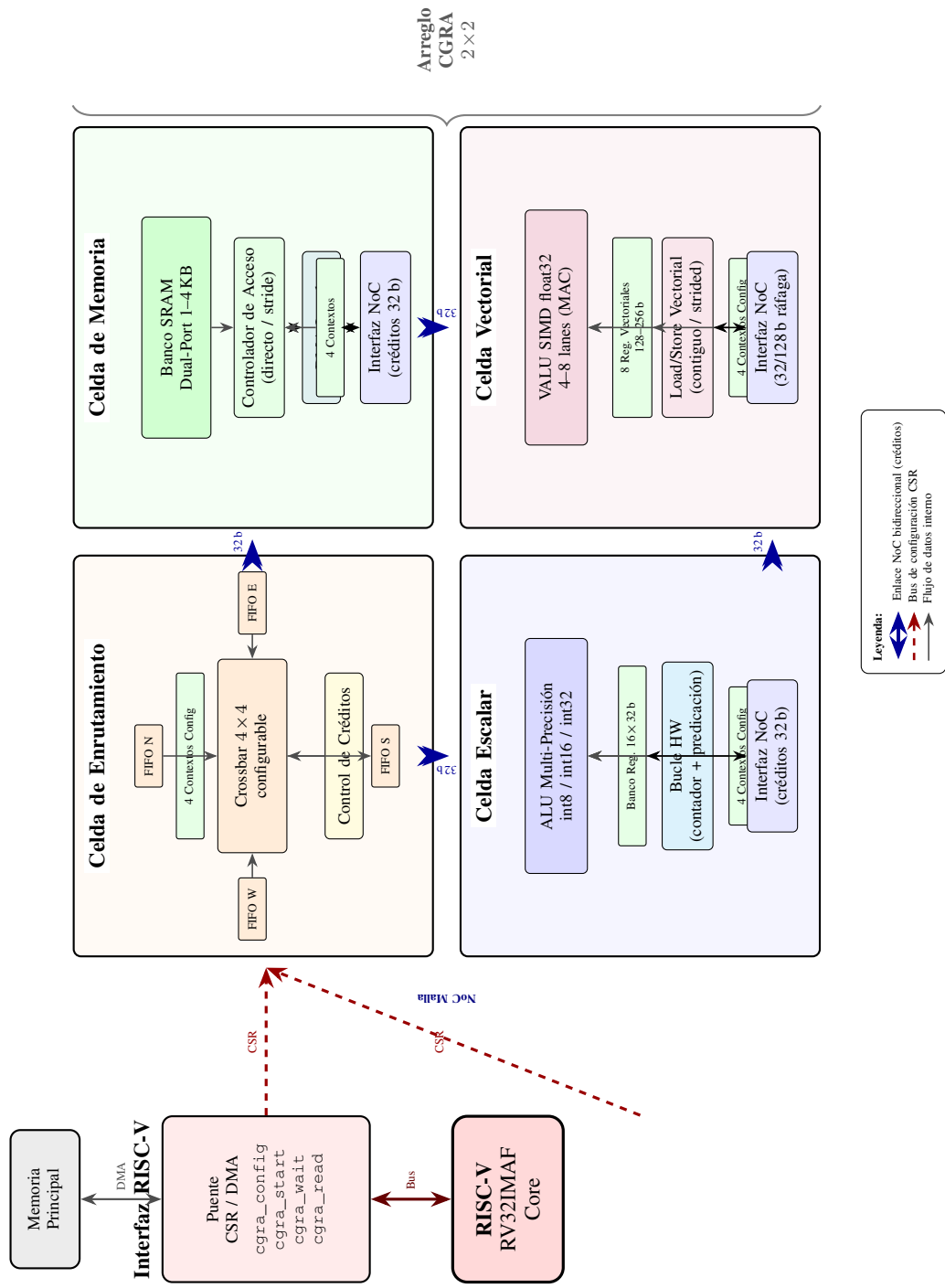


Figura 1. Diagrama de arquitectura de segundo nivel del CGRA heterogéneo 2×2 .

Esta distribución de memoria elimina la contención en un único controlador centralizado y permite que las celdas de cómputo adyacentes accedan a datos con latencia predecible y mínima.

IV-B3. Celda de Procesamiento Escalar (Scalar Processing Cell): La celda escalar está optimizada para operaciones aritméticas y lógicas sobre datos enteros con soporte de múltiples precisiones. Sus componentes son:

- **ALU multi-precisión:** Unidad aritmético-lógica que soporta operaciones en modos int8, int16 e int32, incluyendo suma, resta, multiplicación, operaciones lógicas (AND, OR, XOR, shift) y comparaciones. En modo int8, la ALU puede realizar hasta 4 operaciones SIMD empaquetadas por ciclo.
- **Banco de registros local:** Archivo de registros de 8–16 entradas $\times$ 32 bits para almacenamiento temporal de operandos y resultados intermedios.
- **Unidad de control de bucles hardware:** Contador de iteraciones con soporte de predicación que permite ejecutar bucles internos sin overhead de instrucciones de salto, reduciendo la penalización reportada en [12].
- **Registro de configuración multi-contexto:** Almacena hasta 4 configuraciones de operación (opcode, fuentes, destinos, modo de precisión), permitiendo cambio de función en 1 ciclo de reloj.
- **Interfaz NoC:** Puertos de datos de 32 bits con protocolo de créditos para recepción de operandos y envío de resultados a celdas vecinas.

IV-B4. Celda de Procesamiento Vectorial (Vector Processing Cell): La celda vectorial proporciona capacidad de cómputo en punto flotante de alta densidad para cargas de trabajo que requieren operaciones de tipo SIMD. Sus componentes incluyen:

- **Unidad aritmética vectorial (VALU):** Procesador SIMD de 4 u 8 *lanes* operando en float32 (IEEE 754 single precision). Soporta las operaciones: suma, resta, multiplicación, multiplicación-acumulación (MAC) y operaciones de reducción (sum-reduce, max-reduce).
- **Banco de registros vectoriales:** Archivo de 8 registros vectoriales de 4×32 bits (128 bits por registro) o 8×32 bits (256 bits) según la configuración de lanes.
- **Unidad de carga/almacenamiento vectorial:** Interfaz para acceso a datos desde la celda de memoria vecina con soporte de acceso contiguo y *strided*, optimizada para patrones regulares de matrices.
- **Registro de configuración multi-contexto:** Similar a la celda escalar, almacena hasta 4 configuraciones que definen la operación vectorial, el ancho de vector activo y los patrones de acceso.
- **Interfaz NoC extendida:** Puertos de datos de 32 bits (serializados) o 128 bits (modo ráfaga) para transferencia eficiente de vectores completos entre celdas.

Se estima que la celda vectorial float32 consumirá aproximadamente $3 \times$ más recursos DSP que la celda escalar int32,

pero entregará $\sim 4 \times$ más throughput en operaciones de punto flotante, consistente con los *trade-offs* reportados en [?].

IV-B5. Red en Chip (NoC) — Router de Malla: El router NoC interconecta las cuatro celdas en topología de malla 2×2 y provee la interfaz hacia el procesador host RISC-V. Sus características son:

- **Topología:** Malla 2D regular con enlaces bidireccionales de 32 bits entre celdas adyacentes.
- **Control de flujo:** Basado en créditos por enlace, con profundidad de FIFO de entrada de 4 flits. Cada celda mantiene un contador de créditos por vecino para garantizar operación sin pérdida.
- **Arbitraje:** Round-robin entre los puertos de entrada cuando múltiples fuentes solicitan el mismo puerto de salida en el mismo ciclo.
- **Latencia:** 1 ciclo de reloj por salto (*hop*) en el caso sin contención.
- **Interfaz host:** Un puerto adicional conecta la malla al bus del procesador RISC-V (RV32IMAF) mediante un puente CSR/DMA que permite: (a) escritura de registros de configuración de cada celda, (b) transferencia de datos vía DMA entre memoria principal y celdas de memoria distribuida, y (c) lectura de resultados.

IV-B6. Interfaz de Acoplamiento RISC-V: El acoplamiento con el procesador host RISC-V se realiza mediante:

- **Registros CSR dedicados:** Un espacio de direcciones CSR permite al software RISC-V configurar individualmente cada celda del arreglo, seleccionar contextos activos e iniciar/detener la ejecución del CGRA.
- **Motor DMA:** Controlador DMA de un canal que transfiere bloques de datos entre la memoria principal del SoC y las celdas de memoria distribuida del CGRA, liberando al procesador durante las transferencias.
- **Señalización de estado:** Interrupciones y flags de estado que notifican al procesador RISC-V la finalización de operaciones en el CGRA, permitiendo ejecución concurrente host-acelerador.
- **Extensión ISA mínima:** Instrucciones custom (encoding en el espacio custom-0 de RISC-V) para operaciones frecuentes como `cgra_config`, `cgra_start`, `cgra_wait` y `cgra_read`, reduciendo la latencia de control respecto al acceso CSR puro.

Esta interfaz aborda directamente el desafío C4 (latencia de configuración) identificado en las arquitecturas CGRA-RISC [?] y DVHetero [13], reduciendo el overhead mediante carga por DMA y configuración parcial multi-contexto.

IV-B7. Resumen de la Arquitectura de Segundo Nivel: La Tabla IV sintetiza los componentes principales de la arquitectura y sus parámetros clave de diseño.

V. PLAN DE EXPERIMENTACIÓN Y METODOLOGÍA

V-A. Test y estímulo

Para probar un diseño electrónico se necesita un testbench, encargado de dirigir un estímulo hacia el diseño y de facilitar el monitoreo de lo que ocurre en el diseño bajo prueba

Cuadro IV
RESUMEN DE COMPONENTES DE LA ARQUITECTURA DE SEGUNDO NIVEL.

Componente	Función principal	Parámetros clave
Celda de enrutamiento	Interconexión dinámica con buffering	FIFOs 4–8 entradas $\times$ 32 b; crossbar 4×4 ; 4 contextos
Celda de memoria	Almacenamiento local distribuido	SRAM dual-port 1–4 KB; DMA local; latencia 1–2 ciclos
Celda escalar	Cómputo entero multi-precisión	ALU int8/16/32; 16 registros; bucles HW; 4 contextos
Celda vectorial	Cómputo SIMD float32	VALU 4–8 lanes; 8 reg. vectoriales; MAC; 4 contextos
Router NoC	Transporte de datos en malla	32 b enlaces; créditos; 1 ciclo/hop; arbitraje round-robin
Interfaz RISC-V	Acoplamiento host-acelerador	CSR + DMA + ISA custom; RV32IMAF compatible

(DUT). Como se ha hecho en otros diseños, y en los distintos frameworks de desarrollo para CGRA, resulta necesario contar con un algoritmo que la arquitectura sea capaz de ejecutar. Para esto se suele partir de los llamados kernels: algoritmos representativos, típicamente codificados en C (por ejemplo, FFT y GEMM), anotados con directivas de compilación sobre los loops, cuya estructura de flujo de datos puede mapearse sobre el arreglo de PEs de una CGRA. Este es, de hecho, el enfoque que sigue Morpher dentro del ecosistema OpenCGRA: a partir del kernel en C genera el grafo de flujo de datos (DFG) y el layout de memoria scratchpad, y produce los vectores de prueba que alimentan la etapa de simulación/emulación, con los que se verifica la correctitud funcional y se obtienen estimaciones de área y potencia [14]. El uso de FFT como kernel de referencia es además consistente con la literatura de CGRA orientada a procesamiento de señales, donde aparece de forma recurrente junto con rutinas como IDCT, FIR e IIR [2].

Vale la pena notar que incluso en frameworks consolidados como OpenCGRA, sus propios autores señalan que el simulador depende de testbenches escritos manualmente por el usuario y no automatiza la generación de datos de prueba ni las verificaciones de extremo a extremo [14]. Esto refuerza la pertinencia de plantear, desde un inicio, un flujo de pruebas estructurado y progresivo como el que se propone a continuación.

Si bien un objetivo valioso es lograr ejecutar un algoritmo como FFT o GEMM compilado de principio a fin —desde el programa en C hasta el bitstream de configuración y los datos de entrada de la CGRA—, dado el tiempo disponible para el desarrollo del proyecto, el primer objetivo apunta a pruebas algorítmicamente más sencillas. Se proponen los siguientes pasos:

1. **Una sola PE ejecutando $c = a + b$ (constante) o $c = a \times b$.** Esto valida la carga de la palabra de configuración (config word), la lectura de registros/entradas, la operación de la ALU y la escritura del resultado de salida. Es, literalmente, el “hello world” de la arquitectura.
2. **Cadenas de operaciones, en modo pipeline:** suma acumulada ($y = x_0 + x_1 + x_2 + \dots + x_n$) y potencias por cuadrados sucesivos ($x, x^2, x^4, x^8, \dots$).
3. **MAC simple ($acc += a \times b$):** es el bloque base de casi todo kernel real (FFT, GEMM, convoluciones), por lo que si esta prueba funciona correctamente, se gana confianza para escalar hacia diseños más complejos.

Una vez superadas estas pruebas básicas, se puede avanzar hacia benchmarks típicos de la literatura de CGRA, como MachSuite y PolyBench [15], [16], que junto con Wavelib y BEEBS conforman el conjunto de suites utilizado por Morpher para evaluar su flujo de compilación [14]. Estas suites proveen bibliotecas de kernels (incluyendo FFT y GEMM) que pueden mapearse sobre la CGRA mediante un flujo de compilación similar al descrito anteriormente.

V-B. Monitoreo

Dado que este proyecto se desarrollará en SystemC, se busca aprovechar las funcionalidades propias del lenguaje para implementar la medición de desempeño. Se propone implementar un monitor como un componente `SC_MODULE` que observe, de manera no intrusiva, elementos del DUT como señales y contadores internos. Esta separación entre la lógica funcional y la lógica de medición sigue el principio de diseño modular propio de SystemC, en el que el monitor puede añadirse o removerse del testbench sin modificar el módulo bajo análisis.

Se propone que el monitor calcule, como mínimo, las siguientes métricas:

- **Operaciones por segundo (OP/s):** cociente entre el número total de operaciones ejecutadas por el DUT y el tiempo total de ejecución.
- **Latencia promedio:** registrando `sc_time_stamp()` al inicio y al final de cada operación individual, promediando sobre la ventana de muestreo.
- **Tasa de utilización:** proporción de ciclos activos respecto al total de ciclos.

Estas métricas se imprimen a consola mediante `SC_REPORT_INFO`.

Además, cuando el diseño alcance una etapa apta para síntesis en FPGA o ASIC, será posible medir métricas de potencia, área y temporización (*timing*) con las herramientas EDA correspondientes. Esto aplica tanto para la AMD Kria KV260 como para la AMD Alveo U55C.

V-C. Experimentación arquitectónica

Ligado a la exploración del espacio de diseño, se plantea generar variaciones del diseño arquitectónico de la CGRA, modificando la cantidad y el tipo de PEs (*processing elements*). Por ejemplo, cambios en la distribución de tipos de PE (más bancos de memoria, más ALUs de punto fijo, más ALUs de punto flotante), así como distintas dimensiones $n \times n$ del

arreglo, orientadas a optimizar la ejecución de instrucciones vectoriales.

Este tipo de exploración es consistente con la diversidad de arquitecturas CGRA reportada en la literatura, donde el desempeño depende fuertemente de la composición y disposición de los PEs [2].

V-D. Herramientas y Entorno de Desarrollo

Con el fin de asegurar la reproducibilidad estricta de las pruebas de diseño y simulación, se define la infraestructura de herramientas de software con sus respectivas versiones en la Tabla V.

Cuadro V
HERRAMIENTAS DE DESARROLLO Y VERSIONES DEL ENTORNO

Herramienta / Software	Versión	Propósito en el Experimento
SystemC	3.0.1	Modelado y simulación de la arquitectura CGRA.
Xilinx Vivado Design Suite	2025.2	Síntesis lógica y análisis de <i>timing</i> .
ModelSim / Questa Advanced	2025.1	Simulación funcional RTL ciclo a ciclo.

DECLARACIÓN DE USO DE HERRAMIENTAS DE IA

Para este trabajo se hizo uso de Inteligencia Artificial para la ayuda en la estructura de la introducción y el estado del arte, así como para una revisión gramatical y ortográfica del documento.

REFERENCIAS

- [1] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. H. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Computing Surveys*, vol. 52, pp. 1 – 39, 2019.
- [2] K. S. Artur Podobas and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *IEEE ACCESS*, vol. 8, 2020.
- [3] G. Zhu, L. Deng, K. Zhang, W. Fan, B. Jin, W. Cao, F. Zhang, X. Zhou, F. Zhang, and X. Yu, "Dvhetero: A framework for designing and validating heterogeneous soc with risc-v processor and cgra," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, pp. 1 – 30, 2025. [Online]. Available: <https://www.semanticscholar.org/paper/8d8821ac4c9ec42fc45e662ff65a0ccd6361297a>
- [4] EdgeIR Staff. (2024, jan) What is risc-v and why is it important? online. Accessed: 2026-06-11. [Online]. Available: <https://www.edgeir.com/what-is-risc-v-and-why-is-it-important-20240109>
- [5] J. Granja, D. Vázquez, A. Rodríguez, and A. Otero, "Integration of a cgra accelerator with a cva6 risc-v core for the cloud-edge," null. [Online]. Available: <https://www.semanticscholar.org/paper/b4dfe9170dd1bfa04c4eee19794ebe685b388fe0>
- [6] S. Das, K. J. M. Martin, and P. Coussy, "Context-memory aware mapping for energy efficient acceleration with cgras," *Design, Automation & Test in Europe Conference & Exhibition*, vol. null, pp. 336–341, 2019. [Online]. Available: <https://www.semanticscholar.org/paper/b3c002d29c59700cc1827aff29a72b560c86346a>
- [7] E. Del Sozzo, X. Wang, B. Adhi, C. Cortes, J. Anderson, and K. Sano, "Exploration of Trade-offs Between General-Purpose and Specialized Processing Elements in HPC-Oriented CGRA," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*. IEEE, 2024.
- [8] , "ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix," in , 2004, kU Leuven.
- [9] —, "HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect," in , 2017, nTU Singapore.
- [10] —, "Plasticine: A reconfigurable architecture for parallel patterns," in , 2017, stanford.
- [11] —, "Verification of coarse-grained reconfigurable arrays through random test programs," in .

- [12] —, "Loop overhead reduction techniques for coarse grained reconfigurable architectures," in .
- [13] —, "DVHetero: A framework for designing and validating heterogeneous SoC with RISC-V processor and CGRA," in .
- [14] R. Juneja, P. Dangi, T. K. Bandara, Z. Li, D. Wijerathne, L.-S. Peh, and T. Mitra, "Building an open cgra ecosystem for agile innovation," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025, pp. 1–9. [Online]. Available: <https://arxiv.org/abs/2508.19090>
- [15] L.-N. Pouchet, "PolyBench/C: The polyhedral benchmark suite," <https://www.cs.colostate.edu/~pouchet/software/polybench/>, 2012, accessed: 2026-06-18.
- [16] B. Reagen, R. Adolf, Y. S. Shao, G.-Y. Wei, and D. Brooks, "MachSuite: Benchmarks for accelerator design and customized architectures," <https://github.com/breagen/MachSuite>, 2014, gitHub repository. Accessed: 2026-06-18.